LAUNCH READINESS & REMEDIATION PLAN

What is broken, what blocks launch, and how each one is fixed

A working register of the current problems, separated into what must be closed before a restaurant can go live and what can safely follow.

Build `30964dc` · Production · Reference site: Chai Chakhna Company, Kondapur

HOW TO READ THIS

BLOCKER must be closed before any restaurant goes live. **PRE-SCALE** can launch one friendly site without it, but not ten. **FOLLOW-ON** real, not urgent. **VERIFY** unknown to me — needs checking before it can be graded.

Effort estimates assume one developer familiar with the codebase.

Contents

1 The launch question you must answer first
2 Blockers — security & access
3 Blockers — money & compliance
4 Blockers — operational safety
5 Pre-scale problems
6 Unverified items
7 Additional blockers if launching as a POS
8 Phased plan with gates
9 Definition of launch-ready

1 The launch question you must answer first

The blocker list depends entirely on which product you are launching. These are not the same release.

LAUNCHING AS	BLOCKERS	REALISTIC TIMELINE
Management platform + storefront inventory, menu, reports, online ordering	**9 items** §2–§4	**2–4 weeks.** All are contained fixes. No architectural change.
Full POS replacing the till	**9 items + 6 more** §2–§4 and §7	**4–8 months.** Offline-first and hardware are a different application.

RECOMMENDATION

Launch the management platform first. It is weeks away, and the inventory engine is genuinely stronger than the incumbent's. Treat POS as a separate, later, separately-funded product decision.

2 Blockers — security & access

BLOCKER **B1 · Platform admin passwords are `1234` and `12345678`** 1 day

PROBLEM

The panel that creates and suspends every restaurant on the platform is protected by two of the most-guessed passwords in existence. A breach here is not one restaurant — it is all of them.

SOLUTION

Rotate both to generated passwords in a password manager. Add a minimum-strength policy on the platform-admin account, and enable two-factor. Nothing else on this list matters if this is open.

BLOCKER **B2 · No session revocation** 2–3 days

PROBLEM

Tokens last seven days and cannot be invalidated. If a staff phone is lost or an employee leaves, the only remedy is deactivating the whole account, and their existing token still works until it expires.

SOLUTION

Add a `TokenVersion` column per admin, include it in the JWT, and compare on each request in `middleware/Auth.js`. Bumping the number invalidates every session for that user instantly. A "sign out everywhere" control follows for free.

BLOCKER **B3 · Rate limiting does not actually limit** 1 day

PROBLEM

The limiter counts in memory on a serverless instance, so the 20-attempt auth cap is per warm instance, not global. A distributed attempt spread across cold starts is not meaningfully slowed. This is documented honestly in the code but is still not protection.

SOLUTION

Move the store to Upstash Redis via `rate-limit-redis`. The middleware already exists; only the store changes.

3 Blockers — money & compliance

BLOCKER **B4 · GST is hardcoded at one rate for everything** 1–2 weeks

PROBLEM

Every order is taxed 2.5% CGST + 2.5% SGST regardless of item. Real F&B has multiple slabs, and packaging and service charges are taxed differently. There is no item-level tax field, so this cannot be configured — it is a schema gap. Charging the wrong GST is a legal exposure for the restaurant, not just a bug.

SOLUTION

Add `TaxRate` (or a tax-group reference) to `MenuItems`, compute tax per line rather than on the order total, and store the resulting breakdown on the order so historical bills stay reproducible when rates change. Also fix the independent rounding of CGST and SGST, which can make the two components sum to slightly off 5%.

BLOCKER B5 · No void or refund with attribution 1 week

PROBLEM

Orders can be cancelled, but there is no void-with-reason, no partial refund, and no record of who authorised either. Any cash-handling business needs both, and so does an auditor.

SOLUTION

Add a void/refund action requiring a reason and recording the acting admin, following the same append-only pattern the inventory ledger already uses — never mutate the original record.

BLOCKER B6 · Orders do not record who took them 2–3 days

PROBLEM

Every counter order is attributed to "Walk-in Guest" with no staff identity attached. Discounts, cancellations and voids cannot be traced to a person. For an owner this is often the single most valuable thing the system provides.

SOLUTION

Add `CreatedByAdminId` to `Orders`, populate from the authenticated token, and surface it on the order list and detail. Small change, disproportionate value.

4 Blockers — operational safety

BLOCKER **B7 · No staging environment** 2–3 days

PROBLEM

There is no environment to test in. All work today — including schema-affecting changes — was verified against live business data. That is how seven test orders and six negative stock balances ended up in the production ledger, distorting real revenue and valuation figures.

SOLUTION

Create a second Vercel project and database as `staging`, seeded from an anonymised copy. Point CI at it. No release should touch production before passing there.

BLOCKER **B8 · Production data is polluted with test records** 1 day

PROBLEM

Seven test orders sit in live history inflating revenue, and six ingredients carry negative balances (Milk at −123,300 ml) from testing. Reports built on this are wrong today.

SOLUTION

Void the test orders with a reason once B5 exists, and record a physical stock count for the six ingredients via Inventory → Adjustment, which writes proper `ADJUSTMENT` rows rather than editing history.

BLOCKER **B9 · No automated tests on the billing screen** 3–5 days

PROBLEM

`PosOrderBuilder.jsx` (900+ lines) and `PosTableGrid.jsx` have zero test coverage. During one day of work, three visual regressions reached production on that screen — one a syntax error every test passed through and only a build caught, one a clipped status label caught only by a photograph from a real device.

SOLUTION

Cover the order-building path: add item, change quantity, apply options, compute subtotal, place order. Add a layout assertion that no child element escapes its card's bounds, at two viewport widths — that single check would have caught two of the three regressions.

5 Pre-scale problems

Not blocking one careful launch site, but they will bite before ten.

	PROBLEM	SOLUTION	EFFORT
P1	**Client-side pagination everywhere.** Orders fetches every row then slices in the browser; the backend has no `LIMIT`. Fine at 254 orders, unusable at 50,000.	Server-side paging with total counts — the same pattern already applied to Inventory Transactions. Also affects Customers, Menu, Activity Log.	3–4 days
P2	**No low-stock alerting.** Low stock is visible only if someone opens the dashboard.	Daily digest to the owner using the existing notification service.	2 days
P3	**No CSV export.** No way to get data out for accounting or GST filing.	Export on Orders and the report tabs.	2 days

P4	**Order IDs are a global sequence** across tenants, so any restaurant can infer platform-wide volume from their own numbers.	Per-tenant invoice numbering — which GST compliance likely wants regardless.	2–3 days
P5	**An unlimited permanent coupon exists in production.** `CHAI20` has no expiry and no per-customer limit.	Set an expiry and usage limits, and require both when creating a coupon.	1 day
P6	**Free-text ingredient names.** "Refinded Oil" is a live typo referenced by 12 recipes.	Rename it; recipes reference the ID, so it is safe.	1 hour

6 Unverified items VERIFY

I could not confirm these during this work. Each is potentially a blocker and needs checking before it can be graded honestly.

ITEM	QUESTION TO ANSWER
Database backups	Are automated backups enabled, what is the retention, and *has a restore ever been tested?* An untested backup is not a backup. This may well be the largest blocker on the list.
Razorpay mode	Is production running live keys or test keys? A storefront taking real payments through test keys collects nothing.
GST invoice format	Do bills carry the restaurant's GSTIN, a compliant sequential invoice number, and required HSN/SAC detail? This is a legal requirement, not a preference.
Privacy & data retention	You store customer names, phones, emails and addresses across tenants. Is there a privacy policy, a retention period, and a deletion path on request?
Load behaviour	No load testing has been done. Serverless cold starts under a lunch rush are unmeasured.
Sentry	Error reporting was added this session but is DSN-gated. Confirm `SENTRY_DSN` is actually set in production, or nothing is being reported.

7 Additional blockers if launching as a POS

These are on top of §2–§4, and they are the reason the POS timeline is months rather than weeks.

	BLOCKER	SOLUTION	EFFORT
T1	**No offline operation.** Wi-Fi drops, billing stops.	Local-first application (Electron or React Native) with an embedded database and a sync queue. This is a rewrite of the front end, not a change to it.	2–4 months
T2	**No printer or cash drawer.** Browser print dialog is unusable at service speed.	A local ESC/POS print agent is the narrowest viable step and can be built before T1.	3–4 weeks
T3	**No split payment.** Payment is a single enum, so part-cash-part-UPI cannot be recorded.	Replace the field with payment lines against an order.	1 week
T4	**No shift or cash reconciliation.** Nobody can say what the till should contain.	Shift open/close with opening float, expected vs counted cash, and a Z-report.	1–2 weeks
T5	**No KOT lifecycle.** No delta-printing when items are added, no reprint, no cancel-with-reason.	Make KOT a first-class entity with its own lifecycle, distinct from the order.	2 weeks
T6	**No hold, move or split-bill.** Used constantly on a real floor.	Held-order state; move an order between tables; split a bill across payers.	2–3 weeks

8 Phased plan with gates

Phase 0 — Stop the bleeding (this week)

B1 passwords · B8 clean production data · P5 coupon limits · P6 ingredient typo. Confirm the §6 unknowns, starting with **backups and Razorpay mode**.

Gate: no weak credentials, no test data in live reports, backups proven by an actual restore.

Phase 1 — Make it safe to release (2–3 weeks)

B7 staging · B9 POS tests · B2 session revocation · B3 real rate limiting · B6 staff attribution.

Gate: nothing reaches production without passing staging; the billing screen has tests; a lost device can be locked out.

Phase 2 — Make it legal to bill (2–3 weeks)

B4 per-item GST · B5 void and refund · P4 per-tenant invoice numbering · GST invoice format confirmed.

Gate: a chartered accountant reviews a real bill and signs it off.

Phase 3 — Launch the management platform (1 site)

Go live with one friendly restaurant. Watch Sentry, watch reports, fix what real use surfaces. **Do not sell the POS.**

Phase 4 — Scale readiness (3–4 weeks)

P1 server-side paging · P2 low-stock alerts · P3 exports · load testing.

Gate: ten sites will not degrade the platform.

Phase 5 — POS, as a separate decision

Only if funded deliberately. Sequence: T2 print agent → T3/T4/T5/T6 backend capability → T1 offline application. Building T1 first with nothing to run on it is the common failure.

9 Definition of launch-ready

MANAGEMENT PLATFORM — READY WHEN ALL OF THESE ARE TRUE

- No default or weak credentials anywhere; sessions can be revoked.
- A restore from backup has been performed successfully at least once.
- A staging environment exists and every release passes through it.
- The billing screen has automated tests, including a layout-overflow assertion.
- Tax is computed per item and a real bill has been reviewed by an accountant.
- Every order records who created it; voids and refunds require a reason.
- Production contains no test data, and stock balances reflect a physical count.
- Sentry is confirmed receiving events from production.

POS — NOT READY UNTIL ADDITIONALLY

It keeps taking orders with the internet unplugged · it prints a kitchen ticket to a thermal printer · a shift can be opened, closed and reconciled against counted cash · a bill can be split across two payment methods.

Register against build `30964dc`. Effort estimates assume one developer familiar with the codebase and exclude review and release time. Items in §6 are ungraded because they could not be verified during this assessment — several could move to BLOCKER once checked.